



Scala Programming Practical

Practical – 26

Roll No. : S037	Name: Himanshu Shrivastav
Class : SYIT	Date/Time of Submission: 22-09-2026

Practical 27

Implement **one-hot encoding** for a categorical column in a dataset.

Code:

```
object Demo extends App{  
  val data = List("Red", "Blue", "Green", "Red", "Blue")  
  for (value <- data){val r = if (value == "Red") 1 else 0  
    val b = if (value == "Blue") 1 else 0  
    val g = if (value == "Green") 1 else 0  
    println(value + " -> Red:" + r + " Blue:" + b + " Green:" + g)  
  }  
  println("Himanshu Shrivastav")  
}
```

Output:

```
Red -> Red:1 Blue:0 Green:0  
Blue -> Red:0 Blue:1 Green:0  
Green -> Red:0 Blue:0 Green:1  
Red -> Red:1 Blue:0 Green:0  
Blue -> Red:0 Blue:1 Green:0  
Himanshu Shrivastav
```

Practical 28

Compute the Euclidean distance between two Breeze vectors. Use it for nearest neighbor classification.

Code:

```
import breeze.linalg._
```

```

object Demo extends App {
  // Base vectors define कर रहे हैं
  val A = DenseVector(2.0, 3.0)
  val B = DenseVector(5.0, 7.0)

  // A और B के बीच L2 norm (Euclidean distance) कैलकुलेट कर रहे हैं
  val distance = norm(A - B)
  println("Euclidean Distance: " + distance)

  // Target vector C जिसके लिए nearest neighbor ढूँढना है
  val C = DenseVector(2.5, 3.5)

  // Target vector C से A और B की दूरी निकाल रहे हैं
  val d1 = norm(C - A)
  val d2 = norm(C - B)

  // Nearest neighbor check कर रहे हैं
  if (d1 < d2) {
    println("Nearest Neighbor: A")
  } else {
    println("Nearest Neighbor: B")
  }
  print("Himanshu Shrivastav")
}

```

Output:

```

Euclidean Distance: 5.0
Nearest Neighbor: A
Himanshu Shrivastav
Process finished with exit code 0

```

Practical 29

Create a simple Spark MLlib pipeline to classify data. Use logistic regression or decision trees.

Code:

```
import org.apache.spark.sql.Session
import org.apache.spark.ml.feature.VectorAssembler
import org.apache.spark.ml.classification.LogisticRegression

object Demo extends App {

  // SparkSession क्रीएट कर रहे हैं
  val spark = SparkSession.builder()
    .appName("Classification")
    .master("local[*]")
    .getOrCreate()

  import spark.implicits._

  // सैंपल ट्रेनिंग डेटासेट तैयार कर रहे हैं
  val data = Seq(
    (1.0, 0.0),
    (2.0, 0.0),
    (3.0, 0.0),
    (7.0, 1.0),
    (8.0, 1.0),
    (9.0, 1.0)
  ).toDF("marks", "label")

  // 'marks' कॉलम को फीचर्स वेक्टर में कन्वर्ट करने के लिए VectorAssembler का उपयोग
  val va = new VectorAssembler()
    .setInputCols(Array("marks"))
    .setOutputCol("features")

  val d = va.transform(data)

  // Logistic Regression मॉडल डिफाइन कर रहे हैं
  val model = new LogisticRegression()
    .setLabelCol("label")
    .setFeaturesCol("features")

  // मॉडल को डेटा पर ट्रेन कर रहे हैं
```

```

val result = model.fit(d)

// प्रेडिक्शन (Predictions) जनरेट कर रहे हैं
val prediction = result.transform(d)

// रिजल्ट्स को कंसोल पर डिस्प्ले कर रहे हैं
prediction.select("marks", "label", "prediction").show()

// Spark सेशन को क्लोज कर रहे हैं
spark.stop()
}

```

Output:

```

+-----+-----+-----+
|marks|label|prediction|
+-----+-----+-----+
|  1.0|  0.0|        0.0|
|  2.0|  0.0|        0.0|
|  3.0|  0.0|        0.0|
|  7.0|  1.0|        1.0|
|  8.0|  1.0|        1.0|
|  9.0|  1.0|        1.0|
+-----+-----+-----+

```